Tailwind Design System Guide

Guidance for the new ITI website • Based on the supplied Figma screenshots, font-file screenshot, and the existing global.css reference

Purpose: keep visual decisions centralized so the site can be tuned by changing a small set of design tokens, rather than editing dozens of page and component classes. This is a design-system guide, not a requirement to copy the previous website's entire styling architecture.

1. Core rule

Use Tailwind CSS for component styling, but define the recurring design decisions as named theme tokens. Components should refer to semantic tokens (for example, a brand background or heading size) instead of repeating raw hex values and arbitrary measurements everywhere. Keep one obvious source of truth for colours, typography, spacing, breakpoints, shadows and radii.

The supplied reference CSS uses Tailwind CSS v4's CSS-first approach: @import "tailwindcss" and an @theme block. Tailwind v4 commonly defines theme values in CSS rather than relying on a large JavaScript tailwind.config.js file. For a new project, ask the chosen tool to use the configuration mechanism supported by the Tailwind version it installs.

2. Visual direction observed in the screenshots

The homepage screenshots suggest a restrained institutional style: dark navy sections, a warm yellow accent, light neutral page backgrounds, white cards, dark readable text, and occasional muted blue-grey surfaces. These are visual estimates from screenshots, not measured Figma colour values. If exact Figma values are available, use those instead of treating the suggestions below as authoritative.

Token role	Initial direction	Usage
Brand / primary	Deep navy (estimate: #202C40)	Header, footer, dark section backgrounds, key structural surfaces
Accent / action	Warm yellow (estimate: #F7B900)	Primary buttons, short section rules, highlighted links and active states
Page background	Very light grey (estimate: #F5F6F8)	Main page background and calm section surfaces
Surface	#FFFFFF	Cards, forms, testimonial panels and content containers
Text primary	Dark navy / charcoal	Headings and primary body text
Text muted	Muted blue-grey	Secondary copy, metadata and helper text
Border	Light cool grey	Input borders, dividers and card outlines

Important: sample the exact colours from Figma if possible. The screenshot is compressed, so do not assume the estimated hex values are pixel-perfect.

3. Suggested token groups

Organize the design system around these groups:

- Colours: brand, accent, page background, surface, primary text, muted text, border, success, error and focus.
- Typography: font families, heading sizes, body sizes, line heights, weights and letter spacing.
- Spacing: a compact, consistent scale for gaps, padding and section spacing.
- Breakpoints: use Tailwind's responsive prefixes consistently; add custom breakpoints only when the design needs them.
- Effects: a small set of shadows, border radii and transitions.
- Layout: reusable max-width/container behaviour and common section padding. These may be CSS variables or reusable utility patterns.

Avoid creating a token for every individual one-off value. Centralize values that repeat or express a design decision; use a one-off utility only when the layout genuinely needs a unique measurement.

4. Tailwind CSS v4 theme pattern

The existing global.css is a Tailwind v4-style CSS entry point. A new project can follow the same general idea without copying its previous brand palette or every old token. Example only:

```
@import "tailwindcss";

/* Register named design tokens with Tailwind v4 */
@theme {
  --color-brand: #202C40;
  --color-accent: #F7B900;
  --color-page: #F5F6F8;
  --color-surface: #FFFFFF;
  --color-text: #202C40;
  --color-muted: #667085;
  --color-border: #DDE2E8;

  --font-heading: "Montserrat", sans-serif;
  --font-body: "Montserrat", sans-serif;

  --breakpoint-xs: 360px;
  /* Keep Tailwind defaults unless Figma requires a change. */
}

/* Optional semantic CSS variables can live here too. */
:root {
  --section-space: 4rem;
}
```

This is a starting pattern, not a final config. The chosen implementation should check the installed Tailwind version and avoid mixing v3 JavaScript-config conventions with v4 CSS-first configuration.

5. How components should consume tokens

Prefer named theme utilities:

```
<section class="bg-brand text-white">
  <h2 class="font-heading text-3xl font-semibold">Section heading</h2>
  <p class="text-muted">Supporting copy</p>
  <a class="bg-accent text-brand" href="/contact">Contact us</a>
</section>
```

The utility names above illustrate intent; ensure contrast and actual token names match the final theme. Avoid repeatedly writing arbitrary values such as bg-[#202C40] or text-[#F7B900] throughout components. When the brand colour changes, changing one token should update every component using that token.

6. Typography and Hindi/English content

The supplied font-file screenshot lists these local font files:

- hind-v17-latin-300.woff2
- hind-v17-latin-500.woff2
- hind-v17-latin-600.woff2
- hind-v17-latin-regular.woff2
- montserrat-v30-latin-300.woff2
- montserrat-v30-latin-500.woff2
- montserrat-v30-latin-600.woff2
- montserrat-v30-latin-regular.woff2

The screenshot proves the filenames are available to the user; it does not establish that they are already in the new project's public folder. When adding them, use local @font-face declarations and correct relative URLs. Avoid downloading fonts at runtime from a third-party URL.

Hind is a plausible Hindi-friendly family and Montserrat is listed as the Latin/brand family in the screenshot. The user plans to add the Hindi font later, so the implementation should make font switching easy through Tailwind theme tokens or semantic classes, rather than hardcoding font-family in individual components.

```
/* Example: define local font faces after placing files in the project */
@font-face {
  font-family: "Montserrat";
  src: url("/fonts/montserrat-v30-latin-regular.woff2") format("woff2");
  font-weight: 400;
  font-style: normal;
  font-display: swap;
}

@font-face {
  font-family: "Hind";
  src: url("/fonts/hind-v17-latin-regular.woff2") format("woff2");
  font-weight: 400;
  font-style: normal;
  font-display: swap;
}

/* In @theme */
@theme {
  --font-heading: "Montserrat", sans-serif;
  --font-body: "Montserrat", sans-serif;
  --font-hindi: "Hind", sans-serif;
}
```

Add the other available weights (300, 500, 600) with matching font-weight values if they are used. Do not declare a file as a weight it does not contain. Use a semantic class such as font-hindi only for Hindi text where desired, or change --font-body later if the whole site's body font should change. If Hindi glyphs are not covered by the supplied Latin subset, a suitable Devanagari font file will be required; the filename suffix 'latin' suggests these files alone may not cover Hindi script.

7. Typography scale and responsive behaviour

- Use named heading and body styles, ideally through theme tokens or a small set of shared utility patterns.
- Use responsive utilities to match the separate mobile and desktop Figma frames; don't simply shrink the desktop layout.
- Avoid setting a different arbitrary font size in every section. Establish a small scale, then add exceptions only where the design visibly requires them.
- Check line-height and wrapping with both Hindi and English text because glyph widths and line breaks differ.
- Do not assume the screenshot's displayed copy is final; placeholder content should preserve approximate line lengths and density so layout testing remains meaningful.

8. Images and icons

- Use local assets only. Do not pull images from remote URLs.
- For the initial build, use one local placeholder image file in all image positions, while keeping image paths easy to replace later.
- Use local icon assets supplied from Figma when available. Do not fetch icons from icon websites or remote icon APIs.
- Keep icon usage easy to change: prefer local SVG files or a simple central icon component that references local assets.
- Reserve image dimensions or aspect ratios to reduce layout shift; use appropriate loading behaviour for below-the-fold images.

The one-image placeholder instruction is for development only. It should not result in broken image URLs or a requirement for network access.

9. Design-system implementation rules

1. Create one obvious theme entry point and document where tokens live.
2. Use token names based on purpose (brand, accent, surface, text) rather than page names.
3. Avoid repeating raw brand colours, font-family declarations and common spacing values in page markup.
4. Do not overbuild a component library for a fixed 12-page site; reuse shared elements when it clearly reduces duplicated work.
5. Keep the Figma screenshots as the visual source of truth and compare mobile and desktop independently.
6. When a design change is global, update the token rather than patching each component.
7. Check the actual Tailwind version installed before using configuration syntax.

10. Quick review checklist

- Can the main brand colours be changed in one place?
- Can heading/body font choices be changed without editing every page?
- Are spacing, radii and shadows consistent where the design repeats?
- Do mobile and desktop layouts match their respective Figma references?
- Are all images and icons local?
- Do fonts load locally with font-display: swap?
- Do Hindi and English text both wrap and render correctly?
- Are raw arbitrary values limited to true one-off cases?

Source note: the supplied global.css is from the previous project and uses Tailwind CSS v4's @theme tokens. Its burgundy, maroon, sage and cream palette is not the palette shown in the new homepage screenshots, so it should not be copied unchanged.